

codehive

Python Class Methods

Defining Behavior and Object Logic

1. What are Methods?

Methods are functions that exist inside a class. While **Properties** define the state (data) of an object, **Methods** define its behavior (actions).

Every method in a class must have `self` as its first parameter to link the action back to the specific object instance.

2. Basic Method Syntax

Defining a method is just like defining a function, but inside the class indentation block.

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"Hello, I am {self.name}")

p1 = Person("Emil")
p1.greet() # Executes the behavior
```

3. Methods with Parameters

Methods can accept external arguments just like regular functions, allowing them to process data dynamically.

```
class Calculator:
    def add(self, a, b):
        return a + b

calc = Calculator()
res = calc.add(10, 5) # 15
```

Note that `self` is handled automatically by Python; you only provide the arguments `a` and `b`.

4. Modifying State via Methods

Methods are often used to update an object's properties safely. This is a core part of **Encapsulation**.

```
class Person:
    def __init__(self, age):
        self.age = age

    def celebrate_birthday(self):
        self.age += 1 # Modify internal property
        print(f"Now aged: {self.age}")
```

5. The `__str__()` Magic Method

By default, printing an object shows a cryptic memory address. The `__str__()` method lets you define a human-readable string representation.

Input

```
print(p1)
```

Output (with `__str__`)

```
Tobias (36)
```

```
def __str__(self):  
    return f"{self.name} ({self.age})"
```

6. Interaction: Multiple Methods

Classes can contain multiple methods that interact with the same data set, creating a self-contained system.

Example: Playlist Manager

- `add_song()`: Appends to a list via `self`.
- `remove_song()`: Filters the list via `self`.
- `show_playlist()`: Loops through the list via `self`.

7. Method Deletion

Just like properties, methods can be removed using `del`. However, you must delete it from the **Class**, which prevents all objects from using it.

```
del Person.greet
```

If you call `p1.greet()` after this line, a **AttributeError** is raised because the blueprint no longer contains that instruction.

8. Object Orchestration

In high-level Python development, methods are the "API" of your object. Other parts of your program should interact with objects primarily through their methods rather than touching properties directly.

This design pattern is known as **Interface-based design**.

9. Exercise: Fill in the Blanks

Coding Challenge

Goal: Make code return John (36)

```
class Person:
    __init__(self, name, age):
        self.name = name
        self.age = age
    def __str__(self):
        return f'{self.name}({self.age})'
```

10. Mastery Checklist

- ✓ Methods define object actions.
- ✓ Always include `self` as the first argument.
- ✓ Use `__str__` for better debugging visibility.
- ✓ Methods can change `self` properties.
- ✓ Pass external parameters for calculations.

codehive

Functional Object Series v1.4